

WEB TECHNOLOGIES — COURSE PROJECT

FreshRoute

AI-Powered Farm-to-Table Supply Chain Platform

Technologies Required

React · NestJS · PostgreSQL · LLM AI API · WebSocket · JWT Auth

Web Technologies · Full-Stack Track

1. Project Overview

Food waste and supply-demand mismatch cost agricultural cooperatives millions annually. FreshRoute connects small farms directly with restaurants and grocery stores, replacing phone-based ordering with a real-time digital supply chain — and embeds AI to forecast demand, price produce dynamically, and handle quality disputes.

The challenge is not simply building a marketplace. Students must model perishable inventory with time-based spoilage risk, multi-party workflows (farmer → platform → buyer → driver), and AI that operates on time-series and natural-language data simultaneously.

1.1 Learning Objectives

- Model perishable inventory with time-based attributes (harvest date, shelf life, spoilage risk)
- Build a multi-party order workflow with status transitions guarded by role
- Implement NestJS scheduled cron jobs for automated spoilage and delivery alerts
- Integrate AI for quantitative forecasting and qualitative complaint handling
- Build a map-based delivery tracker using a frontend mapping library

1.2 Team Size & Duration

- Team size: 3 – 5 students
- Submission: source code + report + live demo

2. Real-World Scenario

2.1 Client Profile

GreenValley Cooperative represents 80 small family farms supplying 200 restaurants and grocery stores across the region:

- Produce range: 120+ SKUs spanning vegetables, fruits, dairy, and eggs
- Average order turnaround: 48 hours from order to delivery
- 3 delivery drivers handling 15–25 stops per day across the region
- Current food waste rate: ~18% of harvested produce due to demand mismatches

2.2 Problems to Solve

Pain Point	Detail
Demand mismatch	Farmers harvest based on guesswork; 18% of produce rots unsold each week
Manual pricing	Prices are set weekly by committee; no dynamic response to surplus or near-expiry stock
Quality disputes	Buyers reject deliveries with vague complaints; disputes take 5+ days to resolve with no structured record
Route inefficiency	Drivers plan their own routes by memory, averaging 23% more distance than optimal
No forecast visibility	Buyers don't know what's coming next week; farmers don't know what buyers need

3. Core Application Modules

3.1 Multi-Role Authentication

Four roles: Farmer (manages their listings and views orders), Buyer (browses catalogue, places orders, raises complaints), Driver (views delivery assignments, updates stop status), Admin (manages cooperative members, views analytics).

3.2 Produce Catalogue & Inventory

- Farmers create produce listings: name, variety, unit, price, quantity available, harvest date
- System auto-calculates days-since-harvest and a spoilage risk score (low/medium/high) based on produce type shelf life
- High-spoilage-risk items are highlighted in the buyer catalogue and trigger farmer alerts
- Farmers can update quantities and mark items as sold out in real time

3.3 Order Management

- Orders progress: Pending → Confirmed → Packed → In Transit → Delivered / Disputed
- Buyers can add multiple line items from multiple farmers in one order
- Order confirmation triggers a real-time WebSocket notification to the relevant farmer(s)
- Each order line tracks quantity ordered vs quantity delivered (for partial fulfilment)

3.4 Delivery Scheduling

- Admin assigns confirmed orders to drivers and creates delivery runs
- Drivers see their assigned stops on a map with addresses and special instructions
- Driver marks each stop as delivered or failed; buyer is notified instantly via WebSocket
- NestJS cron job sends a daily delivery schedule summary to each driver at 6 AM

3.5 Quality & Traceability

- Buyers submit quality complaints with free-text description and optional photo
- Each complaint is linked to a specific order line and farmer
- Complaint resolution workflow: Submitted → Under Review → Resolved (credit/replace/reject)
- Full traceability: each product traces back to farm, harvest date, and delivery run

3.6 Admin Analytics Dashboard

- Weekly waste rate by produce category and farmer
- Demand forecast accuracy (AI predictions vs. actual orders placed)
- Top 10 buyers by volume and revenue
- Driver delivery success rates

4. Required AI Integrations

All AI features must route through the NestJS AI proxy. No direct LLM calls from the React frontend.

4.1 Demand Forecaster

AI Feature 1

Analyses past order patterns, seasonality signals, and day-of-week trends to predict next-week demand per produce category, helping farmers plan their harvests.

- Input: last 8 weeks of order history per produce category, current week of year
- Output: predicted order volume per category for the coming week with a confidence band (low/medium/high)
- Farmers see the forecast on their dashboard alongside their current inventory
- Fallback: last-4-week rolling average is shown as a static forecast

4.2 Dynamic Pricing Assistant

AI Feature 2

Suggests fair, market-responsive prices based on current supply levels, spoilage risk, and buyer demand signals.

- Input: current stock quantity, days since harvest, shelf life, last 30 days of sale prices, demand forecast
- Output: suggested price with a brief rationale (e.g. 'tomatoes are 40% above forecast demand — price can rise 12%')
- Farmers accept or override the suggestion; accepted suggestions are tracked for accuracy analysis
- Fallback: farmer sees only their own historical price range with no AI suggestion

4.3 Quality Complaint Classifier

AI Feature 3

When a buyer submits a quality complaint, AI categorises the defect and drafts a supplier notification.

- Input: buyer's free-text complaint + produce type + days since delivery
- Output: defect category (packaging / contamination / freshness / wrong item / quantity), severity (minor/major/critical), and a drafted supplier alert message
- Critical complaints automatically escalate to admin notification via WebSocket
- Fallback: complaint is submitted with a manual category dropdown for the buyer to fill

4.4 Route Optimiser Chat (Bonus)

AI Feature 4 — Bonus

Driver describes their planned deliveries in plain text; AI suggests a reordered stop sequence to minimise total travel distance.

- Input: list of delivery addresses and any time constraints the driver specifies
- Output: reordered stop sequence with a brief reason for each change
- Driver can accept reordering or ignore and proceed with original sequence

5. Technical Stack & Constraints

Layer	Technology	Notes
Frontend	React 18 + TypeScript	Vite, React Router v6, Axios
State	React Context + Zustand	No Redux required
Backend	NestJS + TypeScript	Modular architecture, REST + WebSocket
ORM	TypeORM or Prisma	Migrations required
Database	PostgreSQL 15	Dockerised for local dev
Auth	JWT (Passport.js)	Access + refresh token pair
Real-time	Socket.io via NestJS	Rooms per role / user
AI	OpenAI GPT-4o or Anthropic Claude	Via NestJS proxy — no direct FE calls
API Docs	Swagger (auto-generated)	All endpoints documented
Testing	Jest + Supertest	Min. 60% controller coverage

5.1 Mandatory Constraints

- The NestJS AI proxy module must sit between the React frontend and any LLM API. No API keys in frontend code.
- All secrets (DB credentials, AI API keys) must be stored in .env files. No hardcoding.
- At least one AI feature must gracefully degrade when the AI service is unavailable.
- All role-protected API routes must have Jest + Supertest integration tests.
- Swagger documentation must be auto-generated and accurate — not written by hand.
- Docker Compose file must bring up the database and backend with a single command.

6. Deliverables

Deliverable 1 — System Design Document

- Entity-relationship (ER) diagram covering all database entities
- API contract listing all endpoints, methods, request/response shapes
- React component tree showing major components and role-visibility
- AI integration architecture: prompt structure, context management, output handling

Deliverable 2 — NestJS Backend

- All core modules implemented with full CRUD operations
- JWT authentication with role guards on all protected routes
- WebSocket gateway for real-time notifications
- AI proxy module routing all LLM requests
- Swagger docs auto-generated at /api
- Jest + Supertest tests with minimum 60% controller coverage
- Database migrations that run cleanly from scratch

Deliverable 3 — React Frontend

- Role-based dashboards for all user types
- Protected routing with redirect-to-login for unauthenticated users
- AI-powered UI components integrated with backend proxy
- Real-time updates via WebSocket
- Admin analytics dashboard with at least four chart types
- Responsive layout (usable on tablet viewport)

Deliverable 4 — AI Integration Report

- Full prompt templates used for each AI feature
- Explanation of context management strategy
- Description of fallback/degradation behaviour for each AI feature
- At least two prompt iterations documented with rationale
- One paragraph on data privacy implications of the AI features

Deliverable 5 — Live Demo & Reflection Report

- 15-minute live demonstration of two complete end-to-end user flows
- Written reflection (1,500 – 2,000 words): three architectural decisions, tradeoffs, and what you would change

7. Grading Rubric

Criterion	Weight	Key Indicators
Backend architecture & API design	25%	Module separation, guard correctness, Swagger coverage, test pass rate
React frontend — UX & state management	20%	Role-based routing, real-time updates, component design, responsiveness
AI integration quality & prompt engineering	25%	All 3 mandatory AI features functional, prompt quality, fallback handling
Real-world problem coverage	15%	All core modules functional, data integrity, edge-case handling
System design document	10%	ER diagram accuracy, API contract completeness, AI architecture clarity
Demo & critical reflection	5%	Both flows completed live, reflection depth, honest tradeoff analysis
Bonus: AI Feature 4	+5%	Functional bonus AI service integrated into relevant role view

Note: All team members are expected to contribute to all layers. Individual contribution will be assessed via Git commit history and a brief individual viva (5 minutes per student) during Demo Day.

Academic Integrity & Use of AI Tools

Students are permitted to use AI coding assistants (GitHub Copilot, Claude, ChatGPT) as development aids. However:

- All AI-generated code must be understood, reviewed, and documented by the team
- AI tools may not be used to write the reflection report or system design document
- Prompt templates submitted in the AI Integration Report must be your team's original work
- Any code copied verbatim from external sources must be cited in a comments block

Plagiarism between teams will result in a zero for all parties involved. Viva questions are designed to verify individual understanding.

Good luck — build something you are proud of.